

Multithreading

Operations on Thread :-

1. One Thread Single Task :-

- Only one thread is created.
- The thread performs only one task.
- Example task: printing the table of a number.

Example -

```
package multithreading;

public class one_thread_single_task extends Thread {
    public void run()
    {
        table();
    }
    void table()
    {
        System.out.print("Table of 5 is:\n");
        for(int i=1;i<=10;i++)
        {
            System.out.println(5*i);
        }
    }
    public static void main(String[] args) {
        one_thread_single_task t1=new one_thread_single_task();
        t1.start();
    }
}
```

Output –

Table of 5 is:

5
10
15
20
25
30
35
40
45
50

2. One Thread Multiple Task :-

- Only one thread is created.
- The thread performs more than one task sequentially.
- Example tasks: printing a table and checking if a number is positive or negative.

Example –

```
package multithreading;

public class one_thread_multiple_task extends Thread{
    public void run()
    {
        table();
        positiveornegative();
    }
    void table()
    {
        System.out.print("Table of 5 is:\n");
        for(int i=1;i<=10;i++)
        {
            System.out.println(5*i);
        }
    }
    void positiveornegative()
    {
        int num=9;
        if(num>0)
        {
            System.out.println("The Number is Positive");
        }
        else if(num<0)
        {
            System.out.println("The Number is Negative");
        }
        else
        {
            System.out.println("The Number is Zero");
        }
    }
    public static void main(String[] args) {
        one_thread_multiple_task t1=new one_thread_multiple_task()
        t1.start();
    }
}
```

Output –

Table of 5 is:

5

10

15

20

25

30

35

40

45

50

The Number is Positive

3. Multiple Thread One Task –

- Multiple threads are created.
- All threads perform the same single task concurrently.
- Example task: printing the table of a number.

Example –

```
package multithreading;

public class multiple_thread_one_task extends Thread{
    public void run()
    {
        table();
    }
    void table()
    {
        System.out.print("Table of 5 is:\n");
        for(int i=1;i<=10;i++)
        {
            System.out.println(5*i);
        }
    }
    public static void main(String[] args) {
        multiple_thread_one_task t1=new multiple_thread_one_task();
        multiple_thread_one_task t2=new multiple_thread_one_task();
        multiple_thread_one_task t3=new multiple_thread_one_task();

        t1.start();
        t2.start();
        t3.start();
    }
}
```

Output –

```
Table of 5 is:
5
10
15
20
25
30
35
40
Table of 5 is:
5
10
```

15

20

25

30

Table of 5 is:

5

10

15

20

25

30

35

40

45

50

45

35

40

45

50

50

4. Multiple Thread Multiple Task –

- Multiple threads are created.
- Each thread performs a different task concurrently.
- Example: Thread 1 prints numbers, Thread 2 prints letters.

Example –

```
package multithreading;
class test extends Thread{
    public void run()
    {
        System.out.print("Table of 5 is:\n");
        for(int i=1;i<=10;i++)
        {
            System.out.println(5*i);
        }
    }
}

public class multiple_threads_multiple_task extends Thread {
    public void run()
    {
        int num=9;
        if(num>0)
        {
            System.out.println("The Number is Positive");
        }
        else if(num<0)
        {
            System.out.println("The Number is Negative");
        }
        else
        {
            System.out.println("The Number is Zero");
        }
    }

    public static void main(String[] args) {
        multiple_threads_multiple_task t1=new multiple_threads_multiple_task(); //1st Thread
        test t2=new test(); //2nd Thraed

        t1.start();
        t2.start();
    }
}
```

Output –

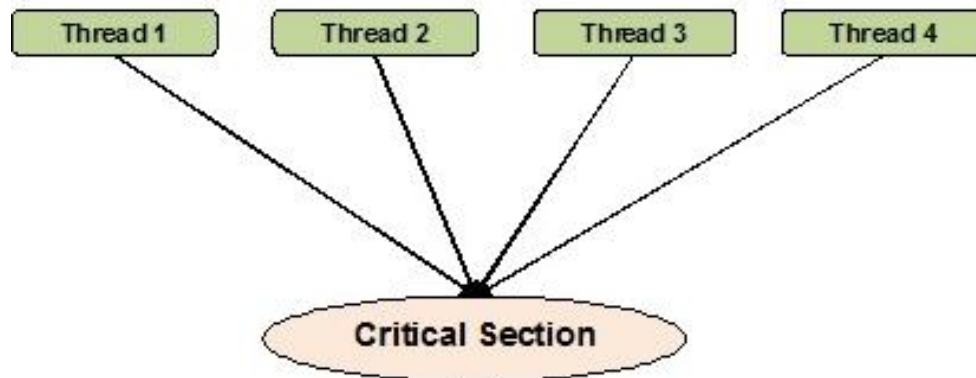
```
The Number is Positive
Table of 5 is:
5
10
15
20
25
```

30
35
40
45
50

Thread Synchronization :-

What is Thread Synchronization?

- It is a mechanism that controls access to shared resources by multiple threads.
- Without synchronization, threads can interfere with each other, causing race conditions.
- Example: Two threads updating a bank account balance simultaneously.



Why is Synchronization Needed?

- To prevent data inconsistency.
- To ensure thread safety when multiple threads access shared resources.

Ways to Achieve Thread Synchronization :-

1) Thread Synchronization Using join() :-

- join() makes the current thread wait until another thread finishes.
- Ensures threads execute in a specific order.

Syntax -

```
thread.join();    // waits indefinitely
thread.join(1000); // waits for 1 second
```

Example –

```
package multithreading;

public class synchronization extends Thread{
    public void run()
    {
        for(int i=1;i<=40;i++)
        {
            if(i%2==0)
            {
                // ... (rest of the code)
            }
        }
    }
}
```



```

        {
            System.out.println(Thread.currentThread().getName()+" :"+i);
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                // TODO Auto-generated catch block
                System.out.println(e);
            }
        }
    }
}

public static void main(String[] args) throws InterruptedException {
    synchronization t1=new synchronization();
    synchronization t2=new synchronization();

    t1.start();
    t1.join(); //wait until thread1 is completing the task
    t2.start();
}
}

```

Output –

```

Thread-0 :2
Thread-0 :4
Thread-0 :6
Thread-0 :8
Thread-0 :10
Thread-0 :12
Thread-0 :14
Thread-0 :16
Thread-0 :18
Thread-0 :20
Thread-0 :22
Thread-0 :24
Thread-0 :26
Thread-0 :28
Thread-0 :30
Thread-0 :32
Thread-0 :34
Thread-0 :36
Thread-0 :38
Thread-0 :40
Thread-1 :2
Thread-1 :4
Thread-1 :6
Thread-1 :8
Thread-1 :10

```

Thread-1 :12
Thread-1 :14
Thread-1 :16
Thread-1 :18
Thread-1 :20
Thread-1 :22
Thread-1 :24
Thread-1 :26
Thread-1 :28
Thread-1 :30
Thread-1 :32
Thread-1 :34
Thread-1 :36
Thread-1 :38
Thread-1 :40

2) Thread Synchronization Using synchronized keyword :-

- synchronized prevents multiple threads from accessing shared resources simultaneously.
- Can be used on methods or blocks.
- Ensures thread safety and prevents race conditions.

Syntax –

```
synchronized returnType methodName(parameters) {  
  
    // critical section  
  
}
```

Example –

```
package multithreading;  
  
public class synchronization1 extends Thread{  
    public void run()  
    {  
        try {  
            even();  
        } catch (InterruptedException e) {  
            // TODO Auto-generated catch block  
            e.printStackTrace();  
        }  
    }  
  
    public synchronized void even() throws InterruptedException  
    {  
        for(int i=1;i<=40;i++)  
        {  
            if(i%2==0)  
            {  
                System.out.println(Thread.currentThread().getName()+" :"+i);  
            }  
        }  
    }  
  
    public static void main(String[] args) throws InterruptedException {  
        synchronization1 t1=new synchronization1();  
        synchronization1 t2=new synchronization1();  
        synchronization1 t3=new synchronization1();  
  
        t1.start();  
        t1.join();    //wait until thread1 is completing the task  
        Thread.sleep(1000);  
        t2.start();  
        t2.join();    //wait until thread2 is completing the task  
        Thread.sleep(1000);  
        t3.start();  
    }  
}
```

Output –

```
|Thread-0 :2  
Thread-0 :4  
Thread-0 :6  
Thread-0 :8  
Thread-0 :10  
Thread-0 :12  
Thread-0 :14  
Thread-0 :16  
Thread-0 :18  
Thread-0 :20  
Thread-0 :22  
Thread-0 :24  
Thread-0 :26  
Thread-0 :28  
Thread-0 :30  
Thread-0 :32  
Thread-0 :34  
Thread-0 :36  
Thread-0 :38  
Thread-0 :40  
Thread-1 :2  
Thread-1 :4  
Thread-1 :6  
Thread-1 :8  
Thread-1 :10  
Thread-1 :12  
Thread-1 :14  
Thread-1 :16  
Thread-1 :18  
Thread-1 :20  
Thread-1 :22  
Thread-1 :24  
Thread-1 :26  
Thread-1 :28  
Thread-1 :30  
Thread-1 :32  
Thread-1 :34  
Thread-1 :36  
Thread-1 :38  
Thread-1 :40
```

Thread-2 :2
Thread-2 :4
Thread-2 :6
Thread-2 :8
Thread-2 :10
Thread-2 :12
Thread-2 :14
Thread-2 :16
Thread-2 :18
Thread-2 :20
Thread-2 :22
Thread-2 :24
Thread-2 :26
Thread-2 :28
Thread-2 :30
Thread-2 :32
Thread-2 :34
Thread-2 :36
Thread-2 :38